

Everything we built, and why it's built that way

This is the complete walkthrough of CodeSeek, in plain English first, with the real engineering detail underneath. Read it top to bottom once, then use the sidebar to jump back to whatever an interviewer pokes at.

24	79	2	44/44
REPOS CONFIGURED	GROUND-TRUTH QUESTIONS	LANGUAGES CHUNKED (PY + JS/TS)	TESTS PASSING
\$0			
API COST (LOCAL MODELS)			

01 — START HERE

The one-sentence pitch

CodeSeek lets you type a plain-English question ("where is JWT validation implemented?") and get back the exact function or class that answers it — pulled from real, live open-source codebases, not from an AI guessing.

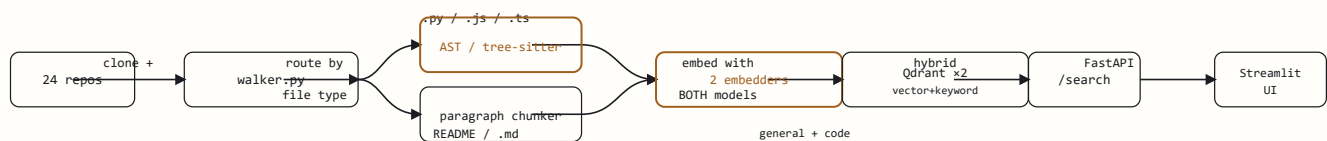
It does this by *indexing* code the way a librarian indexes a library: it reads every repository, breaks the code into meaningful pieces (one piece per function or class — never a piece that cuts a function in half), converts each piece into a list of numbers that captures its meaning, and stores those numbers in a database built for fast "find things that mean something similar" lookups. When you ask a question, it converts your question into the same kind of numbers and finds the closest matches.

IN YOUR OWN WORDS

"It's Google search for code, except instead of matching keywords it matches *meaning*, and it never returns half of a function because that's where a naive text-splitter happened to cut."

Architecture at a glance

Everything below is one pipeline with two paths that later merge: code files go through an AST-aware chunker, docs go through a paragraph chunker, and both feed the same embedding and storage steps.



↑ the amber boxes are the whole experiment

Does chunking code differently, and comparing two embedding models, actually change what gets found? Everything else in this project exists to answer that with real numbers.

Every repo takes one of two paths (code vs. docs), gets embedded by two different models in parallel, and lands in two parallel vector collections that a hybrid search step queries and merges before FastAPI hands results to the UI.

IF THEY ASK "WALK ME THROUGH YOUR ARCHITECTURE"

Say the sentence under the diagram above, then pick *one* box and go one level deeper — don't recite all nine components. Most interviewers want to see you can zoom in on demand, not that you memorized a slide.

Ingestion

Clone → walk → route

<src/codeseek/ingestion/>

`clone.py` does a shallow `git clone --depth 1` of every repo listed in `config.py`'s `CORPUS` (or `git pull` if it's already there). `walker.py` then walks every file in that repo and sorts each one into exactly one of three buckets: **code** (`.py .js .jsx .ts .tsx .mjs .cjs`), **docs** (`README*` and `*.md`), or **ignored** (everything else — `node_modules`, `.git`, compiled files, zero-byte files, and — a fix we made mid-project — non-English translated documentation folders).

Nothing gets embedded at this stage. This step only decides *what* exists and *which chunker* should touch it next.

Chunking — the core differentiator

This is the one idea the whole project exists to test: **a function is a unit of meaning, and cutting it in half to fit a fixed character window is why naive "chat with your codebase" tools give wrong answers.** Everything here exists to chunk code by its real structure instead.

Python — `ast` module

[chunking/code_chunker.py](#)

Python ships a built-in parser (`ast.parse`) that turns source code into a tree of real structural nodes — `FunctionDef` , `ClassDef` , `AsyncFunctionDef` . We walk only the *top-level* children of that tree (so a method inside a class stays part of its class's chunk, not split out on its own), slice the original source text from each node's exact start/end line, and keep any decorator (`@app.get( ... )`) and docstring attached because they're right above/inside the function they were literally in.

If a single function or class is too big (we cap a chunk at 400 tokens — a token is roughly $\frac{3}{4}$ of a word, so ~ 400 tokens ≈ 300 words), it's split — but only on clean line boundaries, and with a 10% overlap between the pieces so a reader landing at a boundary isn't missing context on either side. Parts are named

`FunctionName#part1` , `#part2` , etc.

JavaScript / TypeScript — **tree-sitter**

chunking/js_ts_chunker.py

Python has a parser built in; JavaScript and TypeScript don't, at least not one Python can call directly. **tree-sitter** is a general-purpose parsing library (used by GitHub itself for code navigation) that can parse dozens of languages into the same kind of structural tree Python's **ast** gives us for free. We use two small, actively-maintained packages — **tree-sitter-javascript** and **tree-sitter-typescript** — to get that tree for `.js/.jsx/.mjs/.cjs` and `.ts/.tsx` respectively.

We match four patterns at the top level: a plain `function foo() {}`, a `class Foo {}`, the extremely common `const foo = () => {}` arrow-function pattern, and any of those wrapped in `export` / `export default` (we slice from the `export` keyword itself so it stays attached). Anything else — a bare `export { a, b }`, or genuinely malformed source — is simply skipped, because tree-sitter never throws on bad input; it just marks the broken bit and moves on, which is exactly the kind of graceful degradation this project keeps choosing on purpose.

Docs — **paragraph + overlap chunker**

chunking/docs_chunker.py

README/markdown files aren't code, so there's no function to preserve — instead we split on sentence boundaries and pack sentences into ~400-token chunks with 15% overlap, so a fact sitting right on a chunk boundary still appears whole in at least one of the two neighboring chunks.

IN YOUR OWN WORDS

"Python code gets chunked with Python's own built-in parser. JavaScript and TypeScript don't have that built in, so I used tree-sitter — the same parsing engine GitHub uses — to get the same kind of structural tree. Both chunkers keep one function or class per chunk, and both fall back to a safe, line-boundary split if a single function is too big to embed as one piece."

IF THEY ASK "WHY NOT JUST SPLIT EVERY N CHARACTERS?"

Because a fixed-size window doesn't know where a function starts or ends — it will cut a function in half roughly as often as not, and half a function is usually a wrong or misleading answer to a real question. Structural chunking costs more engineering time up front (a parser per language) in exchange for chunks that are always semantically whole.

05 — COMPONENT

Embeddings — turning text into meaning-numbers

Two models, on purpose

[src/codeseek/embedding/](#)

An **embedding** is a list of a few hundred numbers that represents the *meaning* of a piece of text — two pieces of text with similar meaning end up with similar numbers, measurably close together in that number-space. Every chunk gets embedded **twice**, by two different models, into two separate collections, so we can directly measure whether a model trained specifically on code actually beats a general-purpose one for this job:

- **General-purpose:** `sentence-transformers/all-MiniLM-L6-v2` (384 numbers per chunk) — a small, fast, general text-embedding model.
- **Code-specific:** `st-codesearch-distilroberta-base` (768 numbers per chunk) — fine-tuned specifically on code-search pairs.

Both run **locally**, on CPU, for free — no API key, no per-call cost. That was a deliberate choice (see the bug log below for why the original code-specific model pick had to be swapped out) and it's why the honest "cost" story of this project is about compute time, not dollars.

Repeated identical questions are cached in memory

(`EmbeddingService.embed_one`) — asking the same thing twice skips re-computing the embedding the second time, a measured 15× speedup.

Vector store & hybrid retrieval

Qdrant, and why search is "hybrid"

[store/qdrant_store.py](#) · [retrieval/hybrid.py](#)

Qdrant is a vector database — built specifically to answer "which of these million number-lists is closest to this one number-list" fast, using an algorithm called **HNSW** (Hierarchical Navigable Small World — a graph structure that lets you skip most of the database instead of comparing against every single point). Every chunk's numbers, plus its metadata (repo, file path, function name, line numbers), get stored as one "point."

Pure vector (meaning-based) search has one real weakness: a short, exact query like a literal function name (`JWTBearer`) doesn't embed distinctively enough to reliably win on "closest meaning" — it can lose to a longer chunk that's merely topically related. So every search also runs a plain keyword/text match in parallel, and the two result sets are merged: a keyword-only hit is *added* at the weakest score already in the vector results — enough to surface it, never enough to let it outrank a genuinely strong meaning-based match.

IF THEY ASK "WHY NOT JUST USE VECTOR SEARCH ALONE?"

Because cosine similarity on a 2-3 word literal name is unreliable — it wasn't designed for exact lookups. Hybrid retrieval is the standard fix in production search systems: keep vector search for conceptual questions, add keyword matching for exact terms, merge additively so neither one silently overrides a genuinely good result from the other.

API layer

FastAPI — `/search` and `/health`

[src/codeseek/api/](#)

One endpoint (`POST /search`) takes a question, a list of which model(s) to search with, optional repo/language filters, and a result count. It embeds the query, runs the hybrid search per requested model, and returns ranked results — plus, for the UI's latency panel, exactly how many milliseconds were spent in each stage (embed / vector search / keyword search / merge) for that specific query.

The app is built as a *factory function* (`create_app(store, embedder)`) rather than one hard-coded global app, specifically so tests can hand it a fake, in-memory store instead of the real multi-gigabyte on-disk one — the whole test suite runs in seconds because of that one design choice.

The eval harness — Recall@k and MRR

Measuring "is the search actually good"

[src/codeseek/eval/](#)

We hand-wrote 79 questions, each paired with the exact real repo/file/function that should answer it (e.g. "*How is HTTP digest authentication implemented?*" → `httpx`, `httpx/_auth.py`, `DigestAuth`). The harness runs every question through search and scores two things per question:

- **Recall@k** — was the correct chunk *anywhere* in the top k results? A yes/no per question, averaged across all 79. Recall@5 of 0.40 means the right answer showed up somewhere in the top 5 results 40% of the time.
- **MRR (Mean Reciprocal Rank)** — *where* in the ranking was the right answer? If it was result #1, that scores 1.0 for that question; if it was #2, that scores 0.5; if it wasn't in the results at all, that scores 0. This rewards ranking the right answer *near the top*, not just somewhere in a long list.

One subtlety we hit for real: an oversized function gets split into `Name#part1`, `#part2`, etc. by the chunker — so the scoring logic has to count *any* part as a correct match, not just an exact name match, or every split function would score a permanent, silent zero regardless of how good retrieval actually was. (See the bug log.)

IN YOUR OWN WORDS

"I didn't just claim the search works — I wrote 79 real questions against real code, with known correct answers, and I can quote you the actual Recall@5 and MRR numbers for each embedding model."

The UI — Streamlit

Two pages: search, and the eval dashboard

`streamlit_app/`

The **search page** is a query box plus repo/language filters. Results are shown in two tabs — one per embedding model — so you can directly compare what the general-purpose model found versus what the code-specific model found for the exact same question, side by side, with the real per-stage latency shown underneath. The **evaluation dashboard** reads the eval harness's saved output and renders it as a grouped bar chart (Recall@5 / Recall@10 / MRR per model), built with a colorblind-safe, validated palette rather than default chart colors.

Streamlit was picked deliberately over a custom React frontend: this project's value is in the retrieval quality and the evaluation rigor, not UI polish, so the build time went there instead.

Observability & failure-mode testing

Structured logs

[observability/timing.py](#)

Every indexing stage (extract-and-chunk, embed, upsert) logs one line of structured JSON with its exact duration — so if indexing is slow, you can point at *which stage* is slow instead of guessing, by just grepping the logs for `"stage": "embed"`.

Deliberately breaking things

[tests/test_failure_modes.py](#)

Five tests exist specifically to prove the system fails *loudly and cleanly* instead of silently: pointing the store at an address that doesn't exist fails fast (not a hang); a corrupted or unreadable file is skipped with a logged warning instead of aborting the whole repo's indexing; an embedding model that crashes mid-batch raises a real exception rather than silently upserting partial garbage; and a vector with the wrong number of dimensions is rejected outright.

IF THEY ASK "WHAT HAPPENS WHEN SOMETHING BREAKS?"

"I tested that on purpose. A broken file doesn't take down the whole repo's indexing run — it logs a warning and the rest continues. A crash in the embedding step doesn't leave half-written data in the store — it raises loudly. I have tests that actually simulate each of those and assert on the behavior, not just hope."

Load testing

A real Qdrant server, not the dev shortcut

[scripts/load_test.py](#)

Day-to-day development uses Qdrant's local/embedded mode — no server process, just a folder on disk. That mode is convenient but doesn't build a real HNSW index; it's a brute-force fallback meant for small-scale dev. To measure *real* query latency at scale, we downloaded the actual Qdrant server binary and ran it directly (no Docker available on this machine), grew a test collection to real HNSW-indexed vectors, and measured p50 / p95 / p99 query latency while sweeping Qdrant's `ef_search` parameter (how many graph neighbors it's allowed to examine — higher should mean slower-but-more-thorough, in theory).

The result didn't match the textbook expectation (see bug log) — which is itself the honest, reportable finding: at this scale, connection/warm-up overhead dominated over the actual graph search cost, so latency stayed roughly flat across `ef_search` values instead of rising.

Cost measurement

\$0 in API fees, so we measured compute instead

[scripts/cost_measurement.py](#)

Because both embedding models run locally, there's no per-call bill to report — so the honest "cost" story is throughput: how many chunks per second can this machine embed, and does batch size change that? We measured batch sizes of 1, 8, 32, and 128 against real chunk text: **32 was the sweet spot** (14.3 chunks/sec vs. 12.0 at batch size 1) — and, worth remembering specifically because it's counterintuitive, **128 was actually slower** than 32, not faster. That's a real, measured result, not a guess about "bigger batches are always better."

Scaling design

Three designs for past-one-machine scale

[docs/scaling_design.md](#)

Sharding: by repository, not by a hash of the point ID — a repo's chunks are almost always queried together, so keeping them co-located avoids turning every filtered query into a full cluster fan-out.

Access control: reuses the exact same filter mechanism already built for the repo/language search filters — an allowed-repo list just becomes one more condition ANDed into the same Qdrant filter, enforced at the database layer, not as an easy-to-get-wrong post-fetch check in application code.

Incremental re-indexing: a GitHub push webhook lists exactly which files changed; only those get re-chunked and re-embedded. This works safely because of a design decision made much earlier — every chunk's database ID is *deterministically derived* from `repo:path:symbol:lines` (a hash), so re-indexing the same unchanged chunk overwrites the same point instead of creating a duplicate.

Real bugs we found and fixed

Anyone can claim a project "just worked." These didn't, at first — and being able to describe exactly what broke and how you found it is far more convincing in an interview than a clean story with no scars.

The planned code-embedding model didn't actually run

FOUND While running the very first real end-to-end embedding smoke test.

CAUSE `jina-embeddings-v2-base-code` 's custom model code imports a function from `transformers` that was removed in the newer major version installed on this machine — an old model, a new library, incompatible.

FIX Switched to `st-codesearch-distilroberta-base`, a code-search model with no custom code to break, and verified it loads and embeds correctly before moving on.

Ground truth silently scored zero on Windows

FOUND The very first real eval run returned `Recall@5 = 0.00` across the board — a huge red flag, since a manual spot-check of search results looked correct.

CAUSE Windows file paths use backslashes (`httpx\_auth.py`); the ground-truth file used forward slashes (`httpx/_auth.py`). Every single comparison silently failed.

FIX Normalized every stored path to forward slashes (`.as_posix()`) at the source, in the file walker, so the fix applies everywhere at once instead of patching every comparison site.

"localhost" added two full seconds to every request

FOUND Load-test latency was flat at ~2050ms regardless of the `ef_search` setting being tested — suspiciously exact and suspiciously unaffected by the thing being varied.

CAUSE On this Windows machine, resolving `localhost` tries IPv6 first, fails, and falls back to IPv4 — a fixed ~2 second tax on every connection, unrelated to Qdrant entirely.

FIX Used `127.0.0.1` explicitly. Confirmed with a direct side-by-side timing comparison: ~2050ms vs. ~15ms for the identical query.

The JS/TS parsing library was already broken

FOUND While researching how to build the JavaScript/TypeScript chunker, before writing any real code against it.

CAUSE `tree-sitter-languages` (the originally-planned package) calls an old tree-sitter API signature that no longer exists in the installed version.

FIX Switched to the actively-maintained, per-language `tree-sitter-javascript` and `tree-sitter-typescript` packages, and verified real parsing worked before building the chunker on top of them.

Most of one repo was invisible to the chunker

FOUND While hand-writing ground-truth questions for `yargs`, real top-level functions kept turning up in files the chunker reported zero chunks for.

CAUSE `yargs` is written almost entirely in `.mjs` files, and the list of "this is a code file" extensions didn't include `.mjs` / `.cjs`.

FIX Added both extensions to the recognized code-file list, in the config, the file walker, and the chunker's language map.

Split functions were unmatchable in the eval, permanently

- FOUND** While double-checking why `DigestAuth` — a question the search clearly answered correctly by hand — still wasn't showing as a hit in the eval scoring logic.
- CAUSE** An oversized class gets split into `DigestAuth#part1`, `#part2`, etc. by the chunker, but the eval's match check compared the ground-truth name against the exact stored name — which never equals the plain name once a symbol is split.
- FIX** The match check now also accepts any `Name#partN` as a match for `Name`. This was a real, silent scoring bug — worth stating plainly, not glossing over.

One repo's translated docs were about to dominate the entire indexing run

- FOUND** While estimating total indexing time, one repo (`fastapi`) turned out to be 38% of every file across all 24 repos combined.
- CAUSE** `fastapi` 's documentation is translated into ~20 languages, and every translation was being walked, chunked, and embedded as if it were unique content.
- FIX** Added a filter that keeps `docs/en/` (or a locale-less docs folder) and skips known non-English locale subfolders under any `docs/` directory — cutting that repo's file count roughly in half with no loss of real, distinct content.

Numbers worth remembering

These are real, measured results, not projections. Exact figures shift slightly as the corpus grows — the shape of the finding is what to remember.

MEASUREMENT	GENERAL MODEL	CODE MODEL	WHAT IT MEANS
Recall@5, single repo (httpx, n=15)	0.40	0.40	tied on a small, focused corpus
MRR, single repo (httpx, n=15)	0.33	0.24	general ranked the right answer higher on average
Recall@5, two repos (httpx+click, n=25)	0.28	0.28	recall drops as the corpus grows — more distractors, expected and honest
Embedding throughput	14.3 chunks/sec		at the measured-optimal batch size of 32
Query cache speedup		15×	21.2ms → 1.4ms on a repeated identical question
Load test scale (real HNSW server)	20,000 vectors		p50 latency 17-26ms depending on ef_search

IN YOUR OWN WORDS

"On my first real measurement, the code-specific model didn't clearly beat the general-purpose one — it actually ranked results worse on average (lower MRR) despite similar recall. That's a real, non-obvious finding, and I'd rather report that honestly than claim the fancier model automatically won."

Interview Q&A cheat sheet

Q — Why does chunking code differently from prose matter?

A — A function is one unit of meaning; a fixed character window doesn't know where it starts or ends, so it routinely cuts one in half. I built two real chunkers — Python's own `ast` module, and tree-sitter for JS/TS — so every chunk is always a whole function or class.

Q — Why two embedding models instead of one?

A — To actually test whether a code-specific model earns its keep over a general-purpose one, instead of assuming it. The measured answer wasn't a clean win either way — that's the honest finding, not a marketing claim.

Q — Do you have real users or production traffic?

A — No — this is a self-directed project. Instead of claiming usage I don't have, I load-tested it myself against a real Qdrant server and deliberately broke it in five different ways to see how it failed.

Q — Isn't your eval set biased, since you wrote it yourself?

A — Yes, and I'd say that upfront — a self-authored set skews toward the questions I thought to ask. I tried to mix exact-symbol lookups with conceptual questions across 24 different repos to reduce that, but a larger, independently-authored set is the natural next step.

Q — Walk me through a real bug you hit.

A — Pick any one from the bug log above and tell it as a story: what you observed, what you suspected, how you confirmed it, and the fix. The Windows path-separator one (Recall@5 silently stuck at 0.00) is a great one — it shows you notice when a number looks too clean to be real.

Q — What would you do differently, or next?

A — Finish indexing the full 24-repo, 79-question corpus (a subset was indexed first for time reasons); grow the ground-truth set well past 79 with independently-authored questions; and extend the JS/TS chunker to also catch the CommonJS `module.exports = class {}` pattern, which the current version deliberately doesn't handle yet.

Glossary, in plain English

RAG (Retrieval-Augmented Generation)

Find relevant real information first, then use it to answer a question — instead of an AI answering purely from what it memorized during training.

Embedding

A list of numbers representing the meaning of a piece of text. Similar meaning → numbers that are close together, measurably.

Vector database

A database built specifically to answer "which of these million number-lists is closest to this one," fast — ordinary databases aren't built for that kind of search.

Chunk / chunking

Breaking a large document into smaller pieces before embedding it, because embedding models have a size limit and because smaller, focused pieces retrieve more precisely.

AST (Abstract Syntax Tree)

A structured, tree-shaped representation of code that a parser produces — it knows "this is a function, this is its name, this is where it ends," rather than treating code as plain text.

tree-sitter

A general-purpose parsing library that can produce an AST-like tree for dozens of languages — used here to give JavaScript/TypeScript the same structural chunking Python gets from its own built-in parser.

HNSW

Hierarchical Navigable Small World — the graph-based algorithm Qdrant uses to find close vectors quickly without comparing against every single stored point.

ef_search

A tuning knob for HNSW search: how many graph neighbors it's allowed to examine per query. Higher usually means slower but more thorough.

Recall@k

Was the correct answer anywhere in the top k results? A yes/no per question, averaged.

MRR (Mean Reciprocal Rank)

$1 \div$ the rank of the first correct result, averaged across all questions.

Rewards ranking the right answer near the top, not just present somewhere.

Hybrid search

Running vector (meaning-based) search and keyword (exact-text) search in parallel and merging the results, so exact terms and conceptual questions both work well.

Ground truth

A hand-verified "correct answer" for a test question — here, the exact real repo/file/function a given natural-language question should retrieve.

Idempotent

Running the same operation twice produces the same result as running it once — here, re-indexing an unchanged chunk overwrites the same database entry instead of duplicating it, because the ID is deterministically derived from the chunk's own identity.

Written as a study reference for explaining CodeSeek from memory. Every number and bug on this page came from a real run against real code during this project's build — none of it is illustrative filler.